

1. Make a DNS server and understand the components & changes.

A DNS server (Domain Name System server) is a computer that translates human-readable domain names like google.com into the numerical IP addresses (like 142.250.190.78) that computers actually use to find each other on a network. It works basically like a phone book for the internet — you know the name, the DNS server looks up the number.

Recursive resolver / caching server — this is the kind of DNS server your router, your ISP, or 8.8.8.8 runs. Its job is: when a client asks "what's the IP for google.com?", go figure it out by asking around the internet (root servers → TLD servers → authoritative servers), and remember the answer for a while (cache it) so it's faster next time.

Bind – DNS server software in linux, named (actual daemon)

A DNS server in a lab is just a program (named) that you feed two things: a config file telling it how to behave, and zone files telling it what data to serve. You'll run queries against it with dig and watch how changes to those files change the answers. That feedback loop — edit file, reload, query, see result — is how you actually learn the components.

Steps to follow in order to setup

Step 1: Install BIND

- `sudo dnf install bind bind-utils -y`

bind-utils gives you dig and nslookup, which you'll use constantly to test your server.

Step 2: Look at the default config before touching anything

- `sudo cat /etc/named.conf`

Read through it once. You'll see an options {} block (global behavior) and zone blocks already defined for localhost and reverse lookup — these exist so the server can answer basic loopback queries out of the box. Don't worry about understanding every line yet; just notice the shape: options block, then zone blocks.

Step 3: Make the server listen and accept your queries

Edit /etc/named.conf,

inside options {}:

```
listen-on port 53 { 127.0.0.1; };
```

```
allow-query { localhost; };
```

```
recursion yes;
```

```
forwarders { 8.8.8.8; 1.1.1.1; };
```

This keeps things simple — only your own machine can talk to it for now. You're learning, not building production infra.

Step 4: Start it and prove it works as a resolver

- `sudo systemctl enable --now named`
- `dig @127.0.0.1 google.com`

You should get back an answer with google.com's real IP — because recursion yes + forwarders means your server relayed the question to Google's DNS and cached the result. Run the same dig command again — it'll be near-instant the second time. That's caching, the core concept of a resolver, made visible.

Step 5: Now make it authoritative for a fake domain — this is where the real learning is

Add a zone block to `/etc/named.conf`:

```
zone "lab.test" {  
    type master;  
    file "lab.test.zone";  
};
```

Create the zone file:

- `sudo cp /var/named/named.localhost /var/named/lab.test.zone`
- `sudo vi /var/named/lab.test.zone`

Write this:

```
$TTL 86400
```

```
@ IN SOA ns1.lab.test. admin.lab.test. (  
    2026063001 ; serial  
    3600      ; refresh  
    1800      ; retry  
    604800    ; expire  
    86400 )   ; minimum  
@      IN NS  ns1.lab.test.  
ns1    IN A   127.0.0.1  
web    IN A   192.168.50.10  
mail   IN A   192.168.50.20  
ftp    IN CNAME web.lab.test.
```

Check syntax, then reload:

- `sudo named-checkzone lab.test /var/named/lab.test.zone`
- `sudo systemctl restart named`

Now query your own fake domain:

- `dig @127.0.0.1 web.lab.test`
- `dig @127.0.0.1 ftp.lab.test`
- `dig @127.0.0.1 mail.lab.test`

You'll see your server answering authoritatively (no forwarding involved) with exactly the data you wrote. This is the moment the abstract stuff becomes concrete: SOA is just metadata about the zone, NS says who's in charge, A maps a name to an IP, CNAME is an alias pointing at another name instead of an IP.